МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ

РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №1

з дисципліни «Високорівневі мови програмування та фреймворки»

на тему «Python та Django»

Виконав:

ст. гр. ПЗПІ-23-8

Козлов С. О.

Прийняв:

ст. в. каф. ПІ Саманцов О. О.

Харків 2026

1.1 Мета лабораторної роботи

Ознайомлення з веб-фреймворком Django , вивчення патерну проектування MVC (модель, вигляд, контролер) та практичне застосування моделей, контролерів відображення і шаблонів для створення веб-додатків.

1.2 Індивідуальне завдання (Варіант 5)

- Рівень 1: Написати сайт курсу валют, в якого буде можливість додавати нові курси співвідношення валюти до гривні. Для кожної валюти має бути два значення курсу купівлі та курсу продажу. В результаті отримуємо таблицю із валютами, яка буде мати 3 колонки: назву, курс купівлі та курс продажу.
- Рівень 2: Додайте можливість збереження результатів поточного дня в окремій вкладці.
- Рівень 3: Додайте можливість додавання нових валют за допомогою вбудованою адмін-панелі.
- Рівень 4: Забезпечте доступ по API задля відображення актуального на сьогодні курсу валют.

1.3 Хід вирішення

Повний вихідний код практичної роботи доступний у репозиторії на GitHub за посиланням: `github`

Для реалізації програмного рішення (Варіант 5) було обрано сучасний асинхронний веб-фреймворк FastAPI. Архітектура додатку побудована з дотриманням патерну MVC (Model-View-Controller), що дозволило чітко розділити логіку роботи з даними, серверну маршрутизацію та клієнтський інтерфейс.

1. Розробка моделей даних (Model) В якості інструменту об'єктно-реляційного відображення (ORM) використано бібліотеку SQLAlchemy. База даних реалізована за допомогою двох пов'язаних сутностей:

- Currency (Довідник валют) — містить поля code (наприклад, USD) та name (назва валюти).
- ExchangeRate (Курси обміну) — містить поля buy_rate (курс купівлі), sell_rate (курс продажу), date (дата фіксації курсу) та зовнішній ключ currency_id для зв'язку з таблицею валют (відношення "один-до-багатьох"). Опис моделей виконано з використанням сучасного синтаксису SQLAlchemy 2.0 (клас Mapped та mapped_column).

2. Логіка доступу до даних (CRUD) Для ізоляції бізнес-логіки від контролерів було створено окремі модулі CRUD (Create, Read, Update, Delete). Доступ до бази даних здійснюється асинхронно через AsyncSession. Для оптимізації SQL-запитів та уникнення проблеми "N+1" при завантаженні курсів валют разом з їхніми назвами використовується метод joinedload(ExchangeRate.currency). Функції get_rates_today та get_rates_archive забезпечують вибірку даних за поточну або обрану користувачем дату.

3. Серверна маршрутизація та API (Controller) Контролери реалізовані за допомогою об'єктів APIRouter. Було розроблено повноцінний REST API, який відповідає вимогам Рівня 4 індивідуального завдання:

- Роути GET /currencies/ та POST /currencies/ для роботи з довідником валют.
- Роути GET /rates/today/ (отримання актуального курсу), GET /rates/archive/ (отримання історичних даних для окремої вкладки) та POST /rates/ (додавання нових курсів). Усі вхідні та вихідні дані суворо типізуються та валідуються за допомогою схем Pydantic. Управління сесіями бази даних реалізовано через механізм впровадження залежностей FastAPI (Depends).

4. Клієнтська частина та робота з JSON (View) Для рендерингу базового каркаса сторінок використано шаблонізатор Jinja2 (index.html). Відповідно до вимог практичного завдання, отримання та відображення даних відбувається динамічно на стороні клієнта. На головній сторінці реалізовано клієнтський JavaScript-сценарій, який:

1. За допомогою асинхронного методу fetch() звертається до власного API (/rates/today/).
2. Отримує відповідь від сервера та перетворює її з текстового формату в об'єкт JavaScript за допомогою методу response.json().
3. Динамічно формує DOM-елементи таблиці (рядки <tr> та комірки <td>), заповнюючи їх отриманими даними (назва валюти, курс купівлі та продажу), та монтує їх у підготовлений контейнер.

1.4 Результат виконання

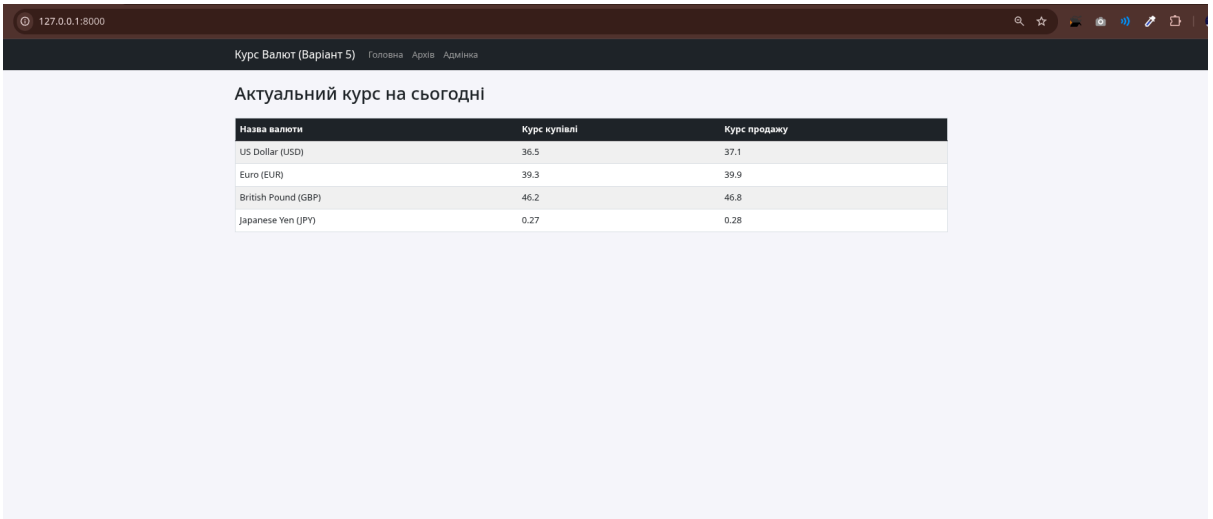

Рисунок 1.1 – Відкрита головна сторінка з таблицею курсів на сьогодні.

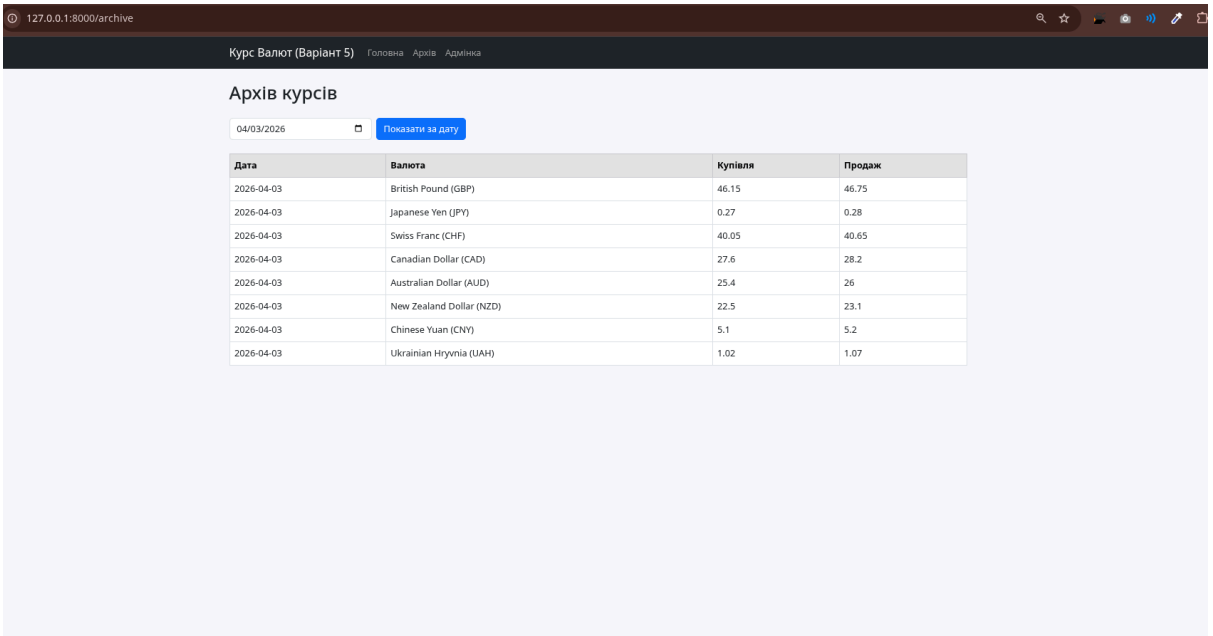

Рисунок 1.2 – Сторінка архіву з обраною датою та результатами за минулий день.

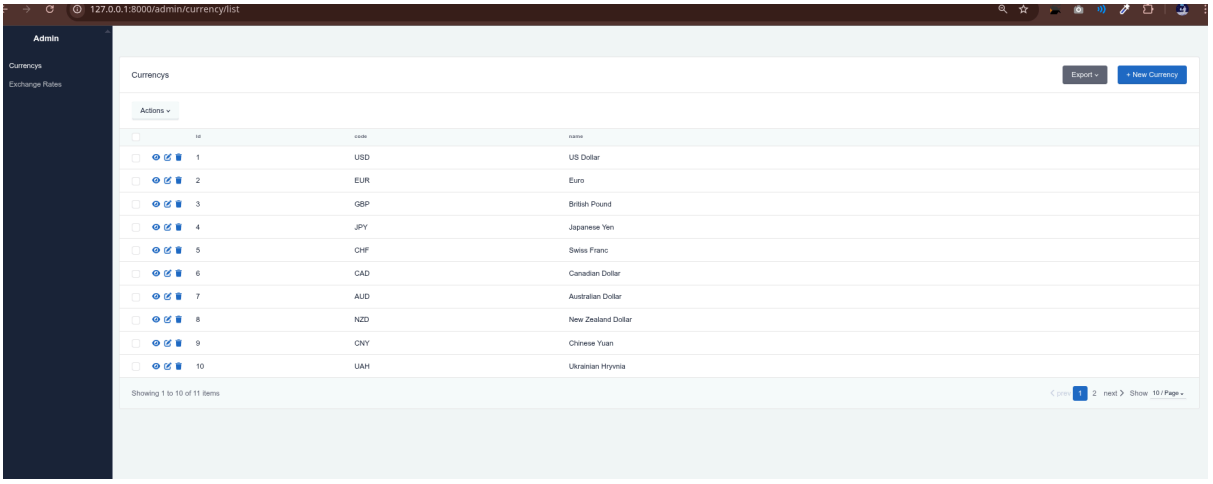

Рисунок 1.3 – Сторінка адмінки

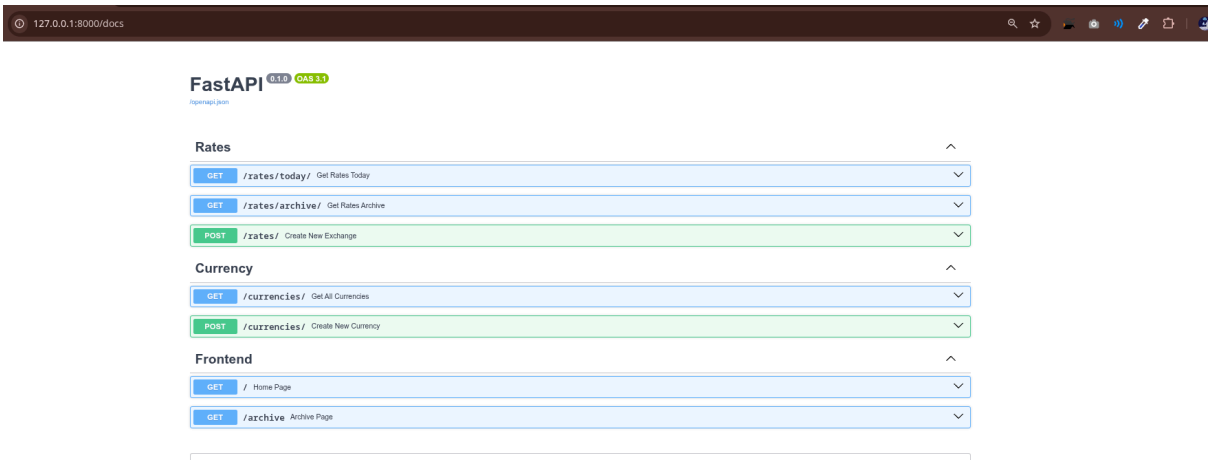

Рисунок 1.4 – Відкритий у браузері Swagger UI

1.5. Висновки

Під час виконання лабораторної роботи було успішно закріплено теоретичні знання та здобуто практичні навички проектування веб-додатків на основі архітектурного патерну MVC. Незважаючи на те, що базовим фреймворком у методичних вказівках є Django, завдання було повністю та успішно реалізовано за допомогою сучасного асинхронного фреймворку FastAPI. Це дозволило глибше зрозуміти принципи побудови REST API та організації клієнт-серверної взаємодії.

Було спроектовано реляційну базу даних (SQLite + SQLAlchemy) для зберігання довідника валют та історії курсів, створено контролери для обробки маршрутів, а також

налаштовано динамічний рендеринг даних на стороні клієнта за допомогою шаблонізатора Jinja2 та асинхронних JavaScript-запитів (fetch()). Усі вимоги індивідуального завдання для Варіанту 5 виконані в повному обсязі, включаючи створення архіву курсів, реалізацію системи адміністрування (через інтеграцію sqladmin) та забезпечення доступу до даних через відкрите API.